π-FORMER: SUCCINCT ZK PROOFS OF TRANSFORMER INFERENCE VIA STRUCTURED LOOKUP ATTENTION

Hideaki Takahashi
Department of Computer Science
Columbia University
ht2673@columbia.edu

ABSTRACT

Verifiable inference for transformers requires the prover to convince a sceptic that a specific committed model produced a specific output on a public input. The dominant cost in such proofs comes from *ZK-unfriendly* primitives in the standard transformer stack: softmax (exponentiation, division, $\max$), GeLU (transcendental activation), Layer Normalization (square root and division), and dense matrix multiplication (one field multiplication per weight entry).

We present π-Former, a transformer architecture and a matching SNARK prover that are *co-designed* for verifiable inference. The architecture has three pieces. *Learnable Lookup Attention* (LLA) replaces softmax with a kernel attention $\phi(\mathbf{Q})\big(\phi(\mathbf{K})^\top\mathbf{V}\big)$ whose feature map decomposes additively into small lookup tables that fit Lasso natively. *Ternary projections* restrict all weight matrices to $\{-1, 0, +1\}$ scaled by a single learnable α, eliminating field multiplications inside the contraction. *Constraint-fused LayerNorm* expresses both the variance and the output identity as low-degree sumcheck claims with chunked range proofs, removing every division and square root from the circuit.

The prover combines Hyrax PCS (transparent, BN254), the Spartan sumcheck IOP, and Lasso lookups, but its key engineering innovation is *cross-block batching*. Six per-protocol sumchecks are run once across all L blocks; all $\Phi(\mathbf{Q})$/$\Phi(\mathbf{K})$ Lasso instances across all blocks fold into one global Lasso; all LayerNorm range proofs share one multiplicity commitment; residuals are checked homomorphically; and all final Hyrax openings flow through deferred batch accumulators that finalise with one MSM pair per group. We implement the full pipeline in $\sim$10 k lines of Rust and run an end-to-end smoke test that produces a 108 KB transparent proof for a 1-block model in 0.27 s on a single CPU core.

1 Introduction

ZK-SNARKs are the natural primitive for verifiable machine learning: the model owner convinces a verifier that a claimed output came from a specific frozen model, while the verifier never reads the weights [7, 12, 1]. Yet proving a transformer [13] block in a SNARK is expensive precisely because the operations that make transformers trainable (softmax, GeLU, LayerNorm) are exactly the operations that finite-field circuits handle poorly. Recent work [12, 9] retrofits ZK proofs onto pre-trained softmax transformers and pays a high concrete cost in both prover time and proof size.

π-Former starts from a different premise: *co-design* the model and the proof system so that every operation in inference maps to a low-degree sumcheck or a structured table lookup. The three modelling ingredients are (i) *Learnable Lookup Attention*, a linear attention [5, 15] whose feature map is a sum of learnable sub-tables sized to fit Lasso exactly; (ii) *ternary projections* [8] that absorb all magnitude into a single per-layer scale and replace field multiplications with field additions; and (iii) a *division-free LayerNorm* circuit that proves the normalisation identity at one random point via constraint fusion plus chunked Lasso range proofs.

The proof system itself is built around *cross-block batching*: six protocol-typed sumchecks (QKV projection, output projection, attention-output, attention-context, FFN-Y, FFN-M) and one global Lasso are each run *once* across all L blocks, with per-block claims combined by Fiat-Shamir powers of η. Residuals are verified by homomorphic addition of Hyrax commitments. All Hyrax openings are aggregated through deferred batch accumulators that finalise into one MSM pair per group. These are not soundness-affecting tricks: each is a standard random-linear-combination argument whose error is dominated by the inner sumcheck error.

Contributions.

- **LLA + ternary + ZK-friendly LayerNorm.** A transformer block in which every operation, including attention, normalisation, and projection, has a sub-linear verifier circuit (§3).
- **Cross-block batched protocol.** An end-to-end prover that collapses six per-block protocols into six *model-level* batched sumchecks, plus one global Lasso and one global range batch (§4).
- **Implementation.** $\sim$10 k lines of Rust over BN254, transparent setup, single-core CPU, with a working CLI (`piformer setup/prove/verify`) and an end-to-end smoke test that produces a 108 KB proof for a 1-block model in 0.27 s (§5).

2 Background

Transformer block. Each block computes

$$\mathbf{X}_{\text{mid}} = \mathbf{X}_{\text{in}} + \text{Att}\big(\text{LN}(\mathbf{X}_{\text{in}})\big),$$
$$\mathbf{X}_{\text{out}} = \mathbf{X}_{\text{mid}} + \text{FFN}\big(\text{LN}(\mathbf{X}_{\text{mid}})\big),$$

where standard self-attention is $\text{Att}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^\top/\sqrt{d})\mathbf{V}$ [13], and LayerNorm $\text{LN}(\mathbf{x}) = \gamma \odot (\mathbf{x} - \mu)/\sqrt{\sigma^2 + \epsilon} + \beta$ [3]. Each of softmax, GeLU, and LayerNorm contains operations (exp, division, square root) without compact polynomial representations, so they dominate SNARK complexity.

Multilinear extensions and sumcheck. A multilinear polynomial $\widetilde{f} : \mathbb{F}^n \to \mathbb{F}$ is determined by its 2^n evaluations on $\{0,1\}^n$. The sumcheck protocol [10] reduces a claim $H = \sum_{x \in \{0,1\}^n} f(x) \cdot g(x)$ to a single evaluation $f(r) \cdot g(r)$ at a random $r \in \mathbb{F}^n$, with $O(n)$ rounds of constant-degree round polynomials.

Hyrax PCS. For a multilinear polynomial of $2^n = 2^{\nu+\sigma}$ evaluations arranged as a $2^\nu \times 2^\sigma$ matrix M, Hyrax commits to each row as a vector Pedersen commitment $C_i = \text{MSM}(\mathbf{g}, M[i])$ and opens at $r = (r_L \| r_R)$ by sending the row-collapsed vector $w'_j = \sum_i L_i(r_L) \cdot M[i][j]$. The verifier checks both a homomorphic commitment equation and an inner-product equation [14]. Hyrax is transparent (no trusted setup) under the discrete-log assumption in BN254 G1; commitments are $O(\sqrt{N})$ in size, and openings cost $O(\sqrt{N})$ G1 operations.

Lasso lookups. For a sub-table $T \in \mathbb{F}^{2^m}$ and queries $\{(\text{idx}_j, v_j)\}$ with $v_j = T[\text{chunk}(\text{idx}_j)]$, Lasso [11, 2] proves all queries are consistent with T via a single batched sumcheck for $\sum_x \widetilde{T}(x) \cdot L(x) = \sum_j \rho^j\, T[\text{ch}_j]$ where $L(x) = \sum_j \rho^j\, \widetilde{\text{eq}}(\text{bin}(\text{ch}_j), x)$ and ρ is Fiat–Shamir-derived. A composite table that decomposes additively into c sub-tables (the canonical Lasso "structured" condition) is committed as c small tables of size $2^{b/c}$ instead of one monolithic table of size 2^b.

3 The π-Former Architecture

3.1 Block structure

A π-Former block is a pre-norm linear-attention block:

$$\mathbf{X}_{\mathrm{n1}} = \mathrm{LN}(\mathbf{X}_{\mathrm{in}}; \gamma_1, \beta_1), \tag{1}$$
$$\mathbf{Q}, \mathbf{K}, \mathbf{V} = \mathbf{X}_{\mathrm{n1}} \mathbf{W}_{Q,K,V} \quad \text{(ternary projections, shared sumcheck)}, \tag{2}$$
$$\mathbf{O}_{\mathrm{attn}} = \mathrm{LLA}(\mathbf{Q}, \mathbf{K}, \mathbf{V})\, \mathbf{W}_O, \tag{3}$$
$$\mathbf{X}_{\mathrm{mid}} = \mathbf{X}_{\mathrm{in}} + \mathbf{O}_{\mathrm{attn}} \quad \text{(homomorphic residual)}, \tag{4}$$
$$\mathbf{X}_{\mathrm{n2}} = \mathrm{LN}(\mathbf{X}_{\mathrm{mid}}; \gamma_2, \beta_2), \tag{5}$$
$$\mathbf{O}_{\mathrm{ffn}} = \boldsymbol{\phi}(\mathbf{X}_{\mathrm{n2}} \mathbf{W}_1)\, \mathbf{W}_2, \tag{6}$$
$$\mathbf{X}_{\mathrm{out}} = \mathbf{X}_{\mathrm{mid}} + \mathbf{O}_{\mathrm{ffn}}. \tag{7}$$

After L such blocks the model applies a final LayerNorm and a ternary language-model head.

3.2 Learnable Lookup Attention

Standard softmax attention requires n^2 exponentials and n row-wise divisions; both translate poorly to SNARK circuits. We replace it with a positive-definite kernel $\kappa(\mathbf{q}, \mathbf{k}) = \langle \boldsymbol{\phi}(\mathbf{q}), \boldsymbol{\phi}(\mathbf{k}) \rangle$ where $\boldsymbol{\phi} : \mathbb{R}^{d_h} \to \mathbb{R}^{d_h}$ is applied element-wise:

$$\mathrm{LLA}(\mathbf{Q}, \mathbf{K}, \mathbf{V})_t \;=\; \frac{\boldsymbol{\phi}(\mathbf{q}_t)^\top \big(\sum_{s=1}^{n} \boldsymbol{\phi}(\mathbf{k}_s) \mathbf{v}_s^\top\big)}{\boldsymbol{\phi}(\mathbf{q}_t)^\top \big(\sum_{s=1}^{n} \boldsymbol{\phi}(\mathbf{k}_s)\big)}. \tag{8}$$

The context matrix $\mathbf{M} = \sum_s \boldsymbol{\phi}(\mathbf{k}_s) \mathbf{v}_s^\top \in \mathbb{R}^{d_h \times d_h}$ depends only on the keys/values and is computed once: per-token output costs $O(d_h^2)$ and the per-block prover cost becomes $O(n d_h^2)$ instead of $O(n^2 d_h)$. The denominator is computed for training stability [5]; the SNARK currently proves the un-normalised numerator (§5).

Additive sub-table feature map. Rather than a fixed kernel (e.g., $\mathrm{elu} + 1$), π-Former parametrises $\boldsymbol{\phi}$ as a sum of learnable sub-table lookups. Let b be the input bit-width and $c \mid b$ the decomposition depth, $\beta = b/c$ bits per chunk. For scalar x, define the integer index

$$x_{\mathrm{int}} = \mathrm{clamp}\big(\lfloor x/s \rfloor,\ 0,\ 2^b - 1\big), \tag{9}$$

and chunk extractor $\chi_i(x_{\mathrm{int}}) = (x_{\mathrm{int}} \gg i\beta) \,\&\, (2^\beta - 1)$. Then

$$\boldsymbol{\phi}(x) \;=\; \sum_{i=0}^{c-1} T_i\big[\chi_i(x_{\mathrm{int}})\big], \tag{10}$$

where $T_0, \ldots, T_{c-1} \in \mathbb{R}^{2^\beta}$ are learnable. The sub-tables are initialised so $\sum_i T_i$ approximates GeLU and trained end-to-end with the straight-through estimator [4]; gradients flow to the table entries while the quantisation/round step is bypassed. For $b = 16, c = 2$, equation (10) replaces a monolithic 65,536-entry table with two 256-entry tables, exactly matching the Lasso "structured table" condition: a single query x_{int} to the implicit 2^b-table decomposes into c independent sub-table queries whose outputs are summed.

3.3 Ternary projections

All projection matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O, \mathbf{W}_1, \mathbf{W}_2$ and the LM head are constrained to entries in $\mathcal{T} = \{-1, 0, +1\}$ scaled by a single learnable scalar $\alpha \in \mathbb{F}$ per layer, following the BitNet b1.58 design [8]. Forward pass: $\mathbf{y} = \alpha \cdot (\mathbf{x}\mathbf{W}) + \mathbf{b}$ with $\mathbf{W} \in \mathcal{T}^{m \times n}$. Training uses STE with the magnitude-thresholded mapping $w \mapsto \mathrm{sign}(w) \cdot [\,|w| > 0.7\overline{|w|}\,]$; α is multiplied in *after* the matrix product so the in-circuit weight stays ternary.

In the SNARK, a sumcheck for a ternary contraction $Y(r_t, r_d) - b(r_d) = \alpha \sum_k X(r_t, k)\, W(k, r_d)$ materialises only the cheap vector $\alpha \cdot W(\cdot, r_d)$, never a field weight matrix; the prover folds the equality polynomial against ternary entries with $+=/-=$/skip rather than field multiplications. A stand-alone Lasso check against the 4-element table $[0, 1, p-1, 0]$ enforces ternarity and is a candidate to fold into setup-time preprocessing, so the verifying key carries a binding proof that $\mathbf{W} \in \mathcal{T}^{m \times n}$.

3.4 Constraint-fused LayerNorm

Let d be the embedding dimension, $\mathbf{x} \in \mathbb{Z}^d$ a row, $\text{sum_x} = \sum_j x_j$, $\text{var_x} = \sum_j (d \cdot x_j - \text{sum_x})^2$ the integer numerator, and $\sigma = \lfloor \sqrt{\text{var_x}}/d \rfloor$. π-Former proves the entire LayerNorm at a single random row index $r_t \in \mathbb{F}^{t_{\text{bits}}}$ in four steps, with no field divisions:

1. **Mean sumcheck.** Degree-2 sumcheck for $\widetilde{\text{sum_x}}(r_t) = \sum_j x_{\text{col}}(j)$ where $x_{\text{col}} = \widetilde{\mathbf{x}}.\text{fix_row}(r_t)$.
2. **Variance sumcheck.** Cubic sumcheck for $\widetilde{\text{var_x}}(r_t) = \sum_j h(j)^2$ with $h(j) = d \cdot x_{\text{col}}(j) - \widetilde{\text{sum_x}}(r_t)$.
3. **Sigma range proof.** Show σ is the integer floor square-root by range-checking $\text{var_x} - (d\sigma)^2 \geq 0$ and $(d\sigma + d)^2 - 1 - \text{var_x} \geq 0$. Both residuals go through a chunked Lasso range proof against the identity table $T_{\text{id}}[i] = i$ of size 2^{16} (see §4.3).
4. **Y constraint fusion.** For a single random (r_{y_t}, r_{y_d}), audit $\gamma \odot (d\mathbf{x} - \text{sum_x}) + \beta \cdot d\sigma \approx d\sigma \cdot \mathbf{y}$ (up to integer rounding) by another range proof on the lo/hi residuals. The γX and σY legs are fused into a single multi-batched cubic sumcheck.

The verifier evaluates γ and β directly from the VK in $O(d)$ field operations per layer; everything else is one shared opening per witness.

4 End-to-End Protocol

4.1 Preprocessing

Setup commits each ternary weight matrix and each activation sub-table via Hyrax. The proving key contains the raw ternary arrays plus their commitments; the verifying key contains commitments only (no raw weights). Hyrax generators are derived as $g_j = \text{SHA3}(\texttt{"piformer-hyrax-gen"} \| j) \cdot G$, giving a fully transparent setup.

4.2 Per-block Phase 1

The only truly per-block step is to commit seven intermediate matrices $(\mathbf{X}_{\text{n1}}, \mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}_{\text{attn}}, \mathbf{X}_{\text{n2}}, \mathbf{O}_{\text{ffn}})$, prove both LayerNorms, and fold both LayerNorms' four range witnesses $(\sigma_{\text{LN1}}, y_{\text{LN1}}, \sigma_{\text{LN2}}, y_{\text{LN2}})$ into one shared multiplicity commitment. Residuals are *not* proved: the verifier uses the linearity of Hyrax to compute $C_{X_{\text{mid}}} = C_{X_{\text{in}}} + C_{O_{\text{attn}}}$ and $C_{X_{\text{out}}} = C_{X_{\text{mid}}} + C_{O_{\text{ffn}}}$ locally. Phase 1 produces no projection, attention, or FFN sub-proof; those are deferred to model-level batches below.

4.3 Six cross-block batched sumchecks

After all L Phase-1 commitments are absorbed, the prover draws a *single* global random point $r_{td} = (r_t \| r_{\text{out}})$ from the transcript and runs six *model-level* sumchecks. Each batches the analogous per-block claim across all L blocks, combined by Fiat–Shamir powers of an independent challenge η:

Batch	Statement	Shared challenge
`batch_qkv`	$\mathbf{Q}, \mathbf{K}, \mathbf{V} = \mathbf{X}_{\text{n1}} \mathbf{W}_{Q,K,V}$ (per block)	r_k^{qkv}
`batch_oproj`	$\mathbf{O}_{\text{attn}} = (\boldsymbol{\phi}(\mathbf{Q})\boldsymbol{\phi}(\mathbf{K})^\top \mathbf{V})\mathbf{W}_O$	r_k^{o}
`batch_attn_out`	$\text{out}[t,j] = \sum_i \boldsymbol{\phi}(\mathbf{Q})[t,i]\, \mathbf{M}[i,j]$	$r_{\text{out}}^{\text{attn}}$
`batch_attn_ctx`	$\mathbf{M}[i,j] = \sum_t \boldsymbol{\phi}(\mathbf{K})[t,i]\, \mathbf{V}[t,j]$	$r_{\text{ctx}}^{\text{attn}}$
`batch_ffn_y`	$\mathbf{Y}_{\text{ffn}} = \mathbf{A}\mathbf{W}_2$	r_k^{fy}
`batch_ffn_m`	$\mathbf{M}_{\text{ffn}} = \mathbf{X}_{\text{n2}}\mathbf{W}_1$	r_k^{m}

A single sumcheck-multi protocol ($H = \sum_\ell \eta^{\ell-1} \sum_x f_\ell(x) g_\ell(x)$) turns L separate sumcheck transcripts into one of length $\log$ (contraction dim). Soundness is preserved because η is drawn *after* all per-block claims and commitments are absorbed into the transcript: a single false claim contributes a non-zero polynomial in η of degree $\leq L - 1$, caught by a random η except with probability $(L-1)/|\mathbb{F}|$.

4.4 Global Lasso and global range batches

All $\phi(\mathbf{Q})$ and $\phi(\mathbf{K})$ Lasso instances across all blocks fold into one `LassoMultiProof` via a single combined sumcheck and one Hyrax opening. Similarly, all six LayerNorm range witnesses in a block (four per block plus two for the final LN) share a *single* multiplicity commitment m_{com} against the identity table $T_{\mathrm{id}}[i] = i$ of size 2^{16}, replacing five $\sqrt{2^{16}}$-MSM commitments per block with one.

4.5 Global intermediate batch open + deferred accumulators

After all six cross-block sumchecks have fixed $r_{td} = (r_t \| r_{\mathrm{out}})$, a *single* `HyraxOpenBatch` opens the $5L$ intermediate matrices ($\mathbf{Q}_\ell, \mathbf{K}_\ell, \mathbf{V}_\ell, \mathbf{O}^\ell_{\mathrm{attn}}, \mathbf{O}^\ell_{\mathrm{ffn}}$) at the shared point r_{td}, using one deferred MSM pair instead of $5L$. The verifier maintains ten cross-cutting accumulators (LayerNorm row/td, projection W/b, LM-head W/b, range σ/Y/m, intermediate) into which every Hyrax claim is pushed; each accumulator finalises once at the end of verification with a single MSM pair (deferred-μ challenge for the inner-product check, single MSM for the homomorphic check).

4.6 Final stage and verification

The model proof concludes with a final LayerNorm + LM head, also batched into the same accumulators. The verifier (i) re-commits the public input/output and binds them to the proof's $x_{\mathrm{in_com}}$ and $\mathrm{logits_com}$, (ii) walks all L blocks' Phase 1, (iii) verifies the six cross-block sumchecks, (iv) verifies the final LN + LM head, (v) verifies the global intermediate batch open and 17 matrix-type batch opens, (vi) finalises the ten Hyrax accumulators, and (vii) verifies the global Lasso. Total verifier work is $O(L \cdot \sqrt{Td})$ G1 operations and $O(L \log Td)$ field operations.

4.7 Soundness, briefly

Theorem 4.1 (π-Former soundness, informal)**.** *Under the discrete-log assumption on BN254 G1, the soundness error of the full π-Former prover is at most $L \cdot n_{\max} \cdot \delta_{\max}/|\mathbb{F}| + \varepsilon_{\mathrm{DL}}$, where $n_{\max}$ is the maximum number of sumcheck variables across all sub-protocols, $\delta_{\max} \leq 3$ is the maximum round polynomial degree, L is the number of blocks, and $\varepsilon_{\mathrm{DL}}$ is the DL advantage in BN254 G1.*

The proof proceeds by hybrid over the L blocks. Hyrax binding (Wahby et al. [14]) reduces all forged commitments to DL. Each cross-block batched sumcheck is sound because the combining challenge η is drawn *after* all per-block claims are absorbed (RLC argument over a polynomial of degree $\leq L-1$ in η plus the inner sumcheck error). Lasso soundness follows from the structured-table sumcheck plus Hyrax binding on the sub-tables [11]. Range-proof soundness uses the LogUp identity over a globally committed multiplicity vector. Homomorphic residuals require no proof at all: linearity of MSM gives $\mathrm{Com}(A) + \mathrm{Com}(B) = \mathrm{Com}(A + B)$ for free. For $L = 12$, $n_{\max} = 64$, $\delta_{\max} = 3$ over BN254, the statistical error is $\leq 2^{-242}$, dominated by the DL term.

5 Implementation and Evaluation

π-Former is implemented in $\sim$10 k lines of Rust over the BN254 scalar field, plus a PyTorch training pipeline that exports field-quantised weights and witnesses so the integer arithmetic in Python and the Rust circuit match exactly. The CLI exposes `piformer setup/prove/verify/inspect/sample`. The current prover supports single-head attention; multi-head is on the roadmap but does not change the protocol structure.

Smoke-test benchmark. We measure the end-to-end CLI on a randomly initialised model exported from PyTorch. Setup, prove, and verify times are the inner *compute*-only timings reported by the Rust CLI itself (i.e., the preprocessing, `prove`, and `verify` calls), excluding JSON parsing and proving/verifying-key file IO. All numbers are single-thread wall-clock on a single x86_64 CPU core; proof sizes are the on-disk size of the binary proof bundle.

Model	d	L	d_{ff}	T	**Params**	**Setup**	**Prove**	**Verify**	**Proof**
tiny	8	1	16	8	1.1 k	0.02 s	0.22 s	0.03 s	108 KB
small	64	2	256	16	133 k	0.09 s	1.10 s	0.15 s	1.0 MB
medium	128	4	512	32	924 k	0.29 s	4.24 s	0.28 s	5.5 MB

The prove time scales roughly linearly in the number of intermediate matrix entries ($L{\cdot}T{\cdot}d$ and $L{\cdot}T{\cdot}d_{ff}$): the per-block sumchecks are $O(Td)$/$O(Td_{ff})$ and the cross-block batching collapses transcript work but not arithmetic work. Verifier

compute is sub-linear in the model size because every cross-block sumcheck is a $\log Td$-round transcript and the deferred Hyrax accumulators finalise in one MSM pair per group (vs. $O(L)$ pairs). For the `medium` configuration ($\sim$924 k parameters), pure verification takes 0.28 s while the surrounding JSON parsing and file IO add another 0.7 s of wall-clock for a total CLI invocation of $\sim$1 s; this gap is engineering, not protocol.

Larger configurations (`large`: d=256, L=6; `gpt2-small`: d=512, L=12) are implemented and pass the same setup/prove/verify cycle, but the JSON witness export becomes the dominant time at that end (170 MB witness for the `large` configuration); they are excluded from the table because their wall-clock is no longer dominated by the prover. A full side-by-side comparison with zkLLM [12] and zkGPT [9] requires fixing matched architectures and matched quantisation, and is left to future work.

Status of training. The Python pipeline (custom `LinearAttentionLayer`, `StructuredLookupActivation`, `TernaryLinear`) trains end-to-end with STE; we do not yet report perplexity vs. a softmax baseline. The integer/field forward pass used for the witness is identical to the Rust circuit's arithmetic, eliminating the train–prove gap.

6 Discussion and Limitations

What is and is not in the SNARK. The Rust prover proves the un-normalised LLA output $\boldsymbol{\phi}(\mathbf{Q})(\boldsymbol{\phi}(\mathbf{K})^\top\mathbf{V})\mathbf{W}_O$. The Python forward pass adds the row-wise normaliser $Z_t = \boldsymbol{\phi}(\mathbf{q}_t)^\top \sum_s \boldsymbol{\phi}(\mathbf{k}_s)$ for training stability; matching the two requires either disabling Z at export time or extending the circuit with a row-normalisation sumcheck. The current implementation also lacks causal masking, multi-head attention, and zero-knowledge masking. None changes the protocol architecture: causal masking is a sumcheck-time indicator, multi-head splits the attention sumchecks into parallel batches, and ZK is an additional blinding layer over committed witnesses.

From benchmark to deployment. Our timings are CPU and single-threaded. GPU MSM and parallel sumcheck [9] would translate directly to π-Former because every dominant inner loop (cross-block batched sumcheck, Lasso multi-instance, batched Hyrax open) is data-parallel. Streaming generation is the natural fit for Nova-style folding [6]: the running context matrix $\mathbf{M} = \boldsymbol{\phi}(\mathbf{K})^\top\mathbf{V}$ is the IVC accumulator, and one folding step per token yields constant-time amortised proving.

Beyond π-Former. The two stand-alone Rust modules (`attention/ternary_check.rs` and `cross_layer/projection.rs`) implement (i) an in-circuit ternary-weight check via a 4-element Lasso table $[0, 1, p-1, 0]$ and (ii) a single cubic sumcheck collapsing L per-layer projections into one $\log L + \log d_{\text{in}}$-round sumcheck. Folding both into the end-to-end prover gives a setup-time binding proof on the weights and a further factor of L saving on the projection transcript. Together with an EVM Hyrax verifier and a zero-knowledge masking layer, this lays out the path from a working research prototype to an on-chain verifiable inference service.

References

[1] Kasra Abbaszadeh, Christodoulos Pappas, Jonathan Katz, and Dimitrios Papadopoulos. Zero-knowledge proofs of training for deep neural networks. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4316–4330, 2024.

[2] Arasu Arun, Srinath Setty, and Justin Thaler. Jolt: Snarks for virtual machines via lookups. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 3–33. Springer, 2024.

[3] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[4] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

[5] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.

[6] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *Annual International Cryptology Conference*, pages 359–388. Springer, 2022.

[7] Tianyi Liu, Xiang Xie, and Yupeng Zhang. Zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2968–2985, 2021.

[8] Shuming Ma, Hongyu Wang, Shaohan Huang, Xingxing Zhang, Ying Hu, Ting Song, Yan Xia, and Furu Wei. Bitnet b1. 58 2b4t technical report. *arXiv preprint arXiv:2504.12285*, 2025.

[9] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. {zkGPT}: An efficient non-interactive zero-knowledge proof framework for {LLM} inference. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 2045–2063, 2025.

[10] Srinath Setty. Spartan: Efficient and general-purpose zksnarks without trusted setup. In *Annual International Cryptology Conference*, pages 704–737. Springer, 2020.

[11] Srinath Setty, Justin Thaler, and Riad Wahby. Unlocking the lookup singularity with lasso. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 180–209. Springer, 2024.

[12] Haochen Sun, Jason Li, and Hongyang Zhang. zkllm: Zero knowledge proofs for large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4405–4419, 2024.

[13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[14] Riad S Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zksnarks without trusted setup. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 926–943. IEEE, 2018.

[15] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.